

PRASIE LAB4

FreeRTOS Tasks

António Barros
July 9, 2026

Exercise: Monitoring System with Gatekeeper and State Protection

Scenario

The system consists of three sensors (Temperature, Humidity, and Pressure).

- **Sensor Tasks:** Execute periodically and update a global data structure.
- **Processing Task:** Analyses the collected data.
- **Logging Task (Gatekeeper):** The only task authorised to write to the display or UART.

Implementation Requirements

1. **Data Structure:** Define a struct that holds the readings for the three sensors.
2. **Mutual Exclusion:** Use a Mutex to protect access to this global struct.
3. **Tasks:**
 - **Task_Sensor (3 instances, Medium Priority):** Each instance simulates reading a specific sensor. Tasks must block access to the struct via the Mutex, update their specific value, and release the lock.
 - **Task_Analyzer (High Priority):** Wakes up periodically, acquires the Mutex, checks if the temperature exceeds a threshold (e.g., 30°C), and sends a message to the Gatekeeper's queue if an alert is triggered.
 - **Task_Gatekeeper (Low Priority):** Remains blocked on an xQueueReceive call. When it receives a message from the Task_Analyzer, it prints the alert to the terminal.

Requirements

- **Restriction:** The Task_Analyzer must not write directly to the screen.
- **Separation of Concerns:** The Mutex must only protect the integrity of the sensor readings. Alarm notifications must be handled strictly via the queue to the Gatekeeper.

Questions to Think About

1. If the Task_Analyzer requires reading all three sensors for a complex calculation, for how long should it hold the Mutex lock, and what is the resulting impact on the latency of the Task_Sensor instances? Can you bound this impact?
2. What would occur if you attempted to acquire the Mutex inside the Task_Gatekeeper while the Task_Analyzer already holds it? How can a deadlock be avoided in this design?